

Algorithmic Optimization & Substring Analysis

COMPLETE MATHEMATICAL DERIVATIONS & JUMPER MECHANICS

Perfect. This is the most important part of the entire algorithm. Everything after this is just implementation. I'll explain it as if we were deriving the algorithm ourselves, not memorizing formulas.

Chapter 3 - Core Observations

Observation 1: We only need two things for every substring

The condition is

$$\text{ones} \geq (\text{zeros})^2$$

Notice something. The entire decision depends on only:

- number of ones
- number of zeros

Nothing else matters. So for every substring we need only these two values.

Observation 2

Suppose Substring: 101101

Instead of counting ones every time, can we already know them? Yes. Using Prefix Sum.

Prefix Sum of Ones

Suppose $s = 101101$

Index	0	1	2	3	4	5
Characters	1	0	1	1	0	1
cntOnes Value	1	1	2	3	3	4

Meaning: `cntOnes[i]` stores the Number of ones from index 0 to i.

Example: `cntOnes[3]` means `1011` contains 3 ones.

Why does Prefix Sum help?

Suppose we want the substring `[2...5]` (which is `1101`).

How many ones?

- **Without prefix:** Scan `1`, `1`, `0`, `1` → Time: `O(length)`
- **With prefix:** Take `cntOnes[5]` which contains 4 ones. Subtract `cntOnes[1]` which contains 1 one. Therefore `4 - 1 = 3`. Done! Only `O(1)`.

Formula 1: For substring `[i...j]`, Number of ones:

$$\text{ones} = \text{cntOnes}[j] - \text{cntOnes}[i-1]$$

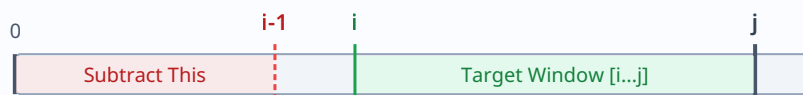
This is exactly your code:

```
int ones = cntOnes[j] - ((i-1 >= 0) ? cntOnes[i-1] : 0);
```

Why subtract?

Picture:

Prefix Sum Subtraction Concept Range Mapping



Suppose `i = 3`. Everything before `i` must disappear. Prefix already counted it. So subtract it.

Observation 3

We don't need another prefix array for zeros. Why? Because:

$$\text{Length} = \text{Zeros} + \text{Ones}$$

$$\text{Zeros} = \text{Length} - \text{Ones}$$

Length is $j - i + 1$. Therefore:

$$\text{zeros} = (j - i + 1) - \text{ones};$$

Why? Example:

Substring: **10110** → Length: 5, Ones: 3. Automatically Zeros = 5 - 3 = 2. No need to count.

Formula 2:

$$\text{zeros} = (j - i + 1) - \text{ones}$$

This is one of the biggest optimizations. Only one prefix array is needed.

Observation 4

Now check $\text{ones} \geq \text{zeros}^2$. Three cases appear:

- **Case 1:** $\text{ones} < \text{zeros}^2$ → Invalid.
- **Case 2:** $\text{ones} == \text{zeros}^2$ → Exactly valid.
- **Case 3:** $\text{ones} > \text{zeros}^2$ → Strongly valid.

Your code handles these three cases separately. That is the entire optimization.

Chapter 4 - Deriving Every Formula

This chapter explains where every formula came from. Nothing is magic.

Formula 1: $\text{ones} = \text{cntOnes}[j] - \text{cntOnes}[i-1]$ (Already derived above.)

Formula 2: $\text{zeros} = (j - i + 1) - \text{ones}$ (Because Length = Zeros + Ones.)

Now comes the interesting part.

Formula 3:

$$\text{skipIdx} = \text{zeros}^2 - \text{ones}$$

This looks mysterious. Let's derive it.

Suppose Current substring is `001`. Zeros: 2, Ones: 1. Need $1 \geq 4$ (False). Missing $4 - 1 = 3$ ones. Exactly $\text{zeros}^2 - \text{ones}$.

Another example: `00111`. Zeros: 2, Ones: 3. Need $3 \geq 4$. Missing 1. Again $4 - 3 = 1$. Exactly $\text{zeros}^2 - \text{ones}$.

Therefore, `skipIdx = zeros * zeros - ones;` means: **How many more ones are required before this substring can possibly become valid?** Nothing more.

Why skip?

Suppose we need 3 more ones. Next character is `0` → Still invalid. Next character is `0` → Still invalid. Until 3 ones appear, there is no point checking. Therefore, Skip. This is the biggest optimization.

Formula 4:

$$\text{shift} = \text{sqrt}(\text{ones}) - \text{zeros};$$

This is harder. Let's derive it. Suppose Current substring is `01111`. Zeros: 1, Ones: 4. Condition $4 \geq 1$ (Valid).

Now, suppose future characters are `111111`. Adding `1` changes $\text{Ones} + 1$, Zeros stay same → Still valid. Again → Still valid. Again → Still valid.

Question: How many extra zeros can we tolerate? Suppose currently $\text{Ones} = 16$. Then $\text{sqrt}(16) = 4$. Maximum zeros allowed is 4. Suppose currently $\text{Zeros} = 2$. We can still increase zeros by $4 - 2 = 2$. Exactly $\text{sqrt}(\text{ones}) - \text{zeros}$.

Meaning `shift` represents: **How many future increases in the zero count can still happen while the substring remains valid?** Your code uses this to jump over several valid substrings together instead of checking each one.

Formula 5:

$$\text{next} = j + \text{shift};$$

Suppose `shift = 4`. Then Current substring and the next 4 positions will still satisfy the condition. Instead of checking `j+1`, `j+2`, `j+3`, `j+4` one by one, your code counts them immediately and jumps.

Why `j += skipIdx - 1`; instead of `j += skipIdx`; ?

Because the `for` loop itself does `j++` after every iteration. So `j += skipIdx - 1`; + `j++` = `skipIdx`. Exactly.

Mathematical Summary

Formula	Meaning
<code>ones = cntOnes[j] - cntOnes[i-1]</code>	Number of ones in the substring
<code>zeros = length - ones</code>	Find zeros without another prefix array
<code>zeros² - ones</code>	How many more ones are needed to become valid
<code>sqrt(ones) - zeros</code>	How many future zero increases can still be tolerated while remaining valid
<code>j += skipIdx - 1</code>	Skip definitely invalid substrings
<code>next = j + shift</code>	Jump across definitely valid substrings

The Entire Thought Process

The algorithm is based on two key mathematical observations:

- If a substring is invalid (`ones < zeros2`), compute exactly how many more ones are missing. Since the condition cannot become true until enough additional ones are included, the algorithm skips checking intermediate endings that cannot possibly work.
- If a substring is already comfortably valid (`ones > zeros2`), compute how much "room" it has before the condition could fail. This lets the algorithm count several future valid substrings in one step instead of verifying each individually.

Important note: These formulas (`skipIdx` and `shift`) are specific to this particular optimized implementation. They are not standard formulas you would derive for every "dominant ones" solution. The official editorial uses a different optimization strategy. Here, these formulas are the mathematical shortcuts that make this implementation efficient.

Chapter 5 - Complete Algorithm & Working

After building the prefix sum of ones, the algorithm generates every possible starting index `i`.

```
for(int i = 0; i < n; i++)
```

Now, for every starting index, it tries different ending indices `j`.

```
for(int j = i; j < n; j++)
```

For every substring `s[i...j]`, we calculate `ones` and `zeros` using:

```
ones = cntOnes[j] - ((i > 0) ? cntOnes[i-1] : 0);  
zeros = (j-i+1) - ones;
```

Now there are only three possible cases.

Case 1: $\text{ones} < \text{zeros}^2$

The substring is invalid.

Example Substring: `001` → zeros = 2, ones = 1. Need `1 ≥ 4` (False).

Instead of checking every next substring one by one, the algorithm computes:

```
skipIdx = zeros*zeros - ones;
```

This tells us: **How many additional ones are required before the substring can possibly become valid.** So we skip:

```
j += skipIdx - 1;
```

Why -1? Because the for loop itself performs `j++` after every iteration. Therefore:

`Current` → `j += skipIdx - 1` → `Loop does j++` → `Total jump = skipIdx`

This avoids checking many obviously invalid substrings.

Case 2: $\text{ones} == \text{zeros}^2$

Example: zeros = 2, ones = 4. `4 ≥ 4`. The substring is exactly valid. Nothing special has to be done. Simply count it.

```
result++;
```

Continue with the next substring.

Case 3: ones > zeros²

The substring is already strongly valid.

Example: zeros = 2, ones = 7. Need $7 \geq 4$. This means we have extra ones.

Instead of checking every future substring separately, the algorithm asks: **How many future extensions are guaranteed to remain valid?** It computes:

```
shift = sqrt(ones) - zeros;
```

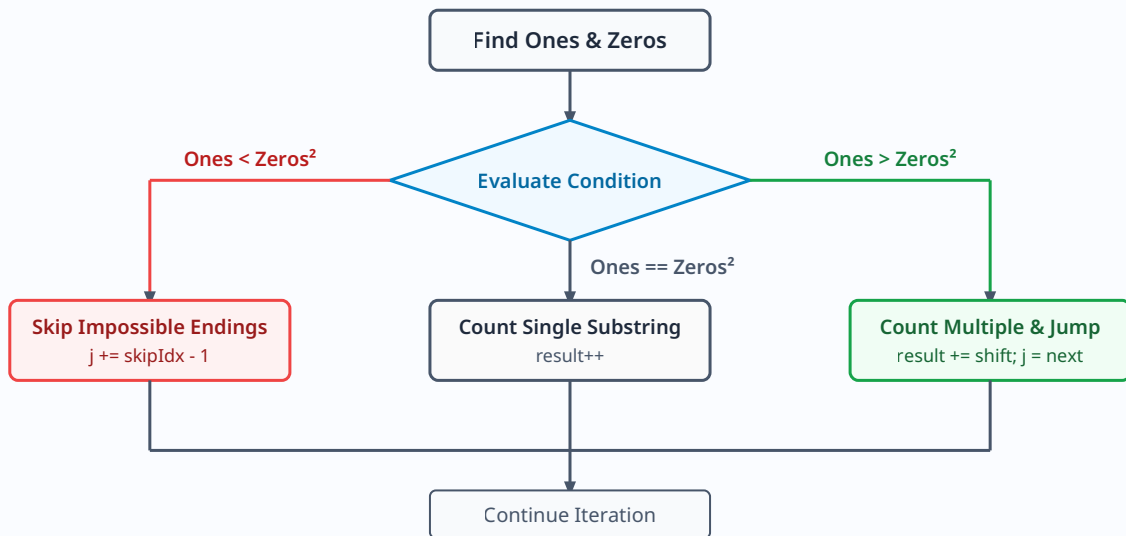
This gives the safe number of future positions that can still satisfy the condition. Now:

```
next = j + shift;
```

Two possibilities occur:

- **next goes outside the array:** Example: $j = 8$, $shift = 5$, $n = 10 \rightarrow next = 13$. Since the jump crosses the end of the string, every remaining substring is guaranteed to be valid. So simply add `result += (n - j - 1);` and then `break;`. No further work is needed.
- **next stays inside the array:** Simply count those guaranteed valid substrings: `result += shift;` and jump `j = next;`. This skips many repeated calculations.

Decision Engine Decision Flow Tree Diagram



Why is this optimization useful?

Without jumping, every substring would be checked individually. With the two mathematical observations (Skip invalid regions & Count multiple valid regions together), the number of iterations is greatly reduced. This is the main optimization of the algorithm.

Chapter 6 - Line-by-Line Code Explanation

- `int n = s.size();` → Store the length of the string.
- `vector<int> cntOnes(n,0);` → Create a prefix sum array. `cntOnes[i]` stores the number of ones from index 0 to i.
- `cntOnes[0]=(s[0]=='1')?1:0;` → Initialize the first position. If the first character is 1, store 1; otherwise 0.
- `for(int i=1;i < n;i++)` → Build the remaining prefix sum.
 - `cntOnes[i]=cntOnes[i-1]+((s[i]=='1')?1:0);` → If the current character is 1, increase the count. Otherwise keep the previous count.
Example: `s = 101101` → Prefix: `1, 1, 2, 3, 3, 4`
- `int result=0;` → Stores the total number of dominant substrings.
- `for(int i=0;i<n;i++)` → Choose every possible starting index.

- `for(int j=i;j<n;j++)` → Choose every possible ending index. Current substring is `[i...j]` .
- `int ones=cntOnes[j]-((i-1>=0)?cntOnes[i-1]:0);` → Compute ones in $O(1)$. Formula: Prefix till `j` - Prefix before `i`.
- `int zeros=(j-i+1)-ones;` → Length = Zeros + Ones → Zeros = Length - Ones.
- `if((zeros*zeros)>ones)` → Current substring is invalid.
- `skipIdx=(zeros*zeros)-ones;` → Compute how many additional ones are required before validity is even possible.
- `j+=skipIdx-1;` → Jump directly. The loop's automatic `j++` will complete the remaining step.
- `else if((zeros*zeros)==ones)` → Substring is exactly valid.
- `result++;` → Count it.
- `else` → Current substring has more ones than required.
- `result++;` → Count the current substring.
- `shift=sqrt(ones)-zeros;` → Estimate how many future extensions can still remain valid.
- `next=j+shift;` → Calculate the furthest position that can be skipped safely.
- `if(next>=n)` → Jump reaches outside the array.
- `result+=(n-j-1);` → Every remaining substring is valid. Count all of them immediately.
- `break;` → No need to continue this starting index.
- `else` → Jump stays inside the array.

- `result+=shift;` → Count all guaranteed valid substrings.
- `j=next;` → Move directly to the next unchecked position. The loop then continues from there.
- `return result;` → Return the total number of dominant substrings.

Code Summary

The algorithm performs the following steps:

1. Build a prefix sum of ones.
2. Generate all possible starting indices.
3. Expand the ending index.
4. Compute ones and zeros in $O(1)$.
5. If the substring is impossible to satisfy, skip several endings.
6. If the substring is exactly valid, count it.
7. If the substring is strongly valid, count several future substrings together and jump ahead.
8. Return the total count.

The entire optimization comes from avoiding unnecessary substring checks through mathematical skipping instead of examining every possible ending individually.

Chapter 7 - Complete Dry Run

Let's dry run the algorithm on a small example to understand how the answer is built.

Input: `s = "1011"` → Length `n = 4`

Step 1: Build Prefix Sum of Ones

Index	0	1	2	3
s[i]	1	0	1	1
cntOnes	1	1	2	3

Meaning: `cntOnes[i]` stores the total number of ones from index 0 to i.

Step 2: Loop Tracing Strategy

i = 0 (Substring starts from index 0)

- **j = 0:** Substring `1` → Length 1, Ones = 1, Zeros = 0. Condition $1 \geq 0^2$ (Valid) → `result = 1`
- **j = 1:** Substring `10` → Length 2, Ones = 1, Zeros = 1. Condition $1 \geq 1^2$ (Valid) → `result = 2`
- **j = 2:** Substring `101` → Length 3, Ones = 2, Zeros = 1. Condition $2 \geq 1$ (Valid) → `result = 3`.
Compute shift: `shift = sqrt(2) - 1` → Integer `sqrt(2) = 1` → `shift = 0` (No jump. Continue normally).
- **j = 3:** Substring `1011` → Length 4, Ones = 3, Zeros = 1. Condition $3 \geq 1$ (Valid) → `result = 4`

i = 1 (Substring begins at index 1)

- **j = 1:** Substring `0` → Length 1, Ones = 0, Zeros = 1. Need `0 ≥ 1` (False). Missing ones: `skipIdx = 12 - 0 = 1`. Jump: `j += 0` (Loop increments automatically next).
- **j = 2:** Substring `01` → Length 2, Ones = 1, Zeros = 1. Condition `1 ≥ 1` (Valid) → `result = 5`
- **j = 3:** Substring `011` → Length 3, Ones = 2, Zeros = 1. Condition `2 ≥ 1` (Valid) → `result = 6`

i = 2 (Substring starts from index 2)

- **j = 2:** Substring `1` → Valid → `result = 7`
- **j = 3:** Substring `11` → Valid → `result = 8`

i = 3 (Substring starts from index 3)

- **j = 3:** Substring `1` → Valid → `result = 9`

Final Answer

9

This dry run shows the overall flow: Compute ones using prefix sum → Compute zeros using length → Check the condition → Either skip invalid regions or count valid substrings.

Chapter 8 - Complexity, Key Observations & Revision Notes

Time Complexity

- **Prefix Sum:** Building the prefix array takes $O(n)$.
- **Nested Loops:** At first glance, it looks like $O(n^2)$. However, the algorithm avoids checking every substring individually by:
 - Skipping substrings that are guaranteed to remain invalid.
 - Counting several guaranteed valid substrings together.

These jumps significantly reduce the number of iterations in practice.

Space Complexity

Only one prefix array is stored: `cntOnes`. Size is $O(n)$. Therefore, $\text{Space} = O(n)$.

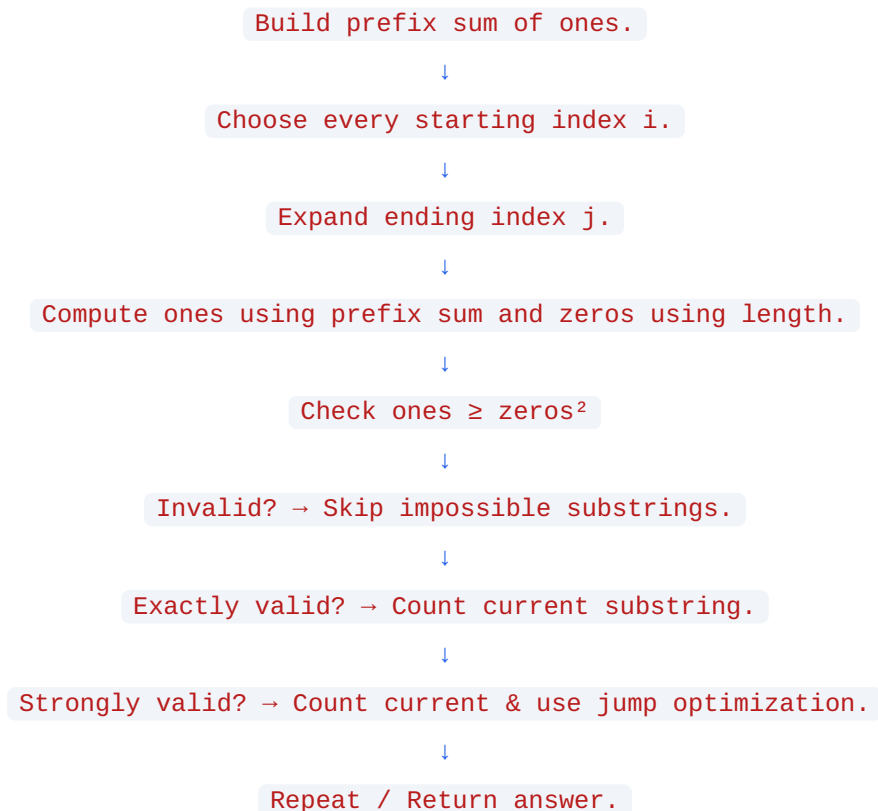
Key Observations

- **Observation 1:** The condition depends only on the Number of Ones and Number of Zeros. No other information about the substring matters.
- **Observation 2:** Prefix Sum allows us to compute the Number of Ones in $O(1)$ instead of scanning the substring.
Formula: `ones = cntOnes[j] - cntOnes[i-1]`
- **Observation 3:** No separate prefix array for zeros is needed. Since $\text{Length} = \text{Ones} + \text{Zeros}$, therefore $\text{Zeros} = \text{Length} - \text{Ones}$.
- **Observation 4:** Three situations exist:
 - **Case 1 ($\text{ones} < \text{zeros}^2$):** Invalid. Skip impossible endings.
 - **Case 2 ($\text{ones} == \text{zeros}^2$):** Exactly valid. Count one substring.
 - **Case 3 ($\text{ones} > \text{zeros}^2$):** Strongly valid. Count the current substring and use the implementation's jump strategy to skip over ranges that are guaranteed to remain valid.

Important Formulas

- **Prefix Ones:** `ones = cntOnes[j] - cntOnes[i-1]`
- **Zeros:** `zeros = (j - i + 1) - ones`
- **Missing Ones:** `skipIdx = zeros2 - ones` → Meaning: Additional ones required before the substring can possibly satisfy the condition.
- **Safe Jump:** `shift = sqrt(ones) - zeros` → Meaning (for this implementation): Estimates how many future extensions can still remain valid, allowing the algorithm to jump ahead instead of checking every ending one by one.

Complete Algorithm Flow Summary



Revision Sheet (1 Minute Summary)

- **Formula 1:** `ones = prefix[j] - prefix[i-1]`
- **Formula 2:** `zeros = length - ones`

- **Cases:**

- `ones < zeros2` → Skip
- `ones == zeros2` → Count 1
- `ones > zeros2` → Count current + Jump ahead

- **What to Remember:** Build the prefix sum of ones. Compute zeros from the substring length. Every substring falls into one of three cases. Use mathematical observations to avoid checking every possible ending. The optimization comes from skipping impossible substrings and jumping across guaranteed valid ones. This transforms a naive brute-force approach into a much more efficient solution.